Università di Pisa

Calcolatori Elettronici

Anno Accademico 2025–2026

Pietro Bacigalupi

5 marzo 2026

Indice

1 Architettura del calcolatore moderno **2**

1.1 Introduzione 2
1.2 Calcolatori Moderni 2
1.3 Lo Spazio di I/O e l'Interazione con il Sistema 3
1.4 Mappatura dello Spazio di Indirizzamento 4
1.5 CPU - Il Ciclo Fondamentale (Fetch-Execute) 4
1.6 L'Evoluzione a 64 bit (AMD64) 5
1.6.1 Organizzazione dei Registri 5
1.7 Indirizzamento Relativo e Limiti Fisici 5
1.8 Spazio di Indirizzamento e Indirizzi Canonici 6
1.9 Allineamento dei Dati in Memoria 6
1.10 Rappresentazione dei dati: Endianness 7
1.11 Nomenclatura e Potenze di 2 8
1.12 Offset, Intervalli e Circolarità 9
1.13 Confini (Boundaries) e Regioni Naturali 9
1.14 L'Interfaccia Fisica: Bus Dati e Byte Enable 10
1.15 L'Organizzazione Fisica della Memoria: Righe e Byte Enable 11
1.16 Analisi di una Lettura Disallineata 12

1 Architettura del calcolatore moderno

1.1 Introduzione

Chi comanda? Il calcolatore è controllato dal **software**, ovvero da una sequenza di istruzioni memorizzate in memoria.

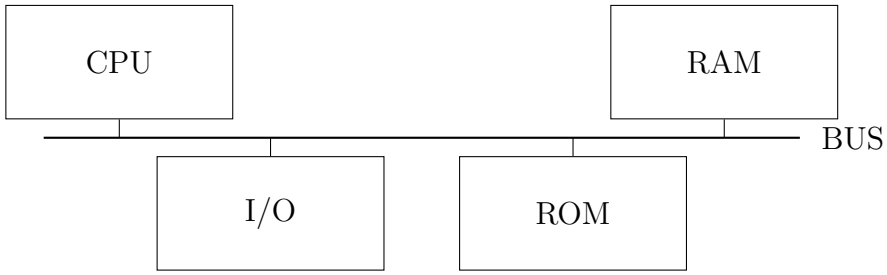
La **CPU** (Central Processing Unit) preleva le istruzioni dalla memoria centrale (RAM), le decodifica e le esegue. Essa non possiede memoria delle operazioni precedentemente eseguite, se non attraverso lo stato dei registri interni.

La CPU, in ogni istante, può accedere esclusivamente:

- ai propri registri interni;
- alla memoria centrale (RAM);
- ai dispositivi collegati tramite il bus.

La **RAM** funge da contenitore temporaneo del programma in esecuzione e dei dati su cui esso opera.

1.2 Calcolatori Moderni



La RAM, infatti, è una sequenza di celle di memoria, ma non contiene metadati sulla natura del proprio contenuto.

Nota Bene: Trasparenza della Memoria

All'interno della memoria non è specificata la natura del dato: non c'è distinzione esplicita tra un'istruzione eseguibile, un numero intero o un indirizzo. È compito della **CPU** interpretare correttamente il dato prelevato in base al contesto (fase di *fetch* dell'istruzione o di *operand fetch*).

Indirizzamento e Strutture Dati Gli indirizzi non sono fisicamente scritti nelle celle di memoria; essi rappresentano la posizione logica e fisica della cella stessa, accessibile tramite il *Address Bus*.

Nelle strutture dati lineari (come gli array), l'efficienza del calcolatore deriva dalla capacità di calcolare rapidamente la posizione di un elemento. Per accedere a un dato è sufficiente conoscere:

1. **Indirizzo Base** (B): il punto di inizio della struttura dati in memoria.
2. **Indice dell'elemento** (i): la posizione relativa.
3. **Dimensione del dato** (S): lo spazio occupato da ogni singolo elemento (es. 4 byte per un *integer*).

Il calcolo dell'indirizzo fisico A dell'elemento i-esimo avviene tramite una semplice operazione aritmetica, estremamente veloce per la CPU:

$$A = B + (i \times S)$$

Questa proprietà garantisce un tempo di accesso costante ($O(1)$) a qualsiasi elemento della struttura, indipendentemente dalla sua posizione.

1.3 Lo Spazio di I/O e l'Interazione con il Sistema

L'interazione tra la CPU e il mondo esterno è mediata dal software. Nessun componente hardware può operare in modo autonomo senza una logica di controllo definita.

Il processo di Bootstrap All'accensione (*Power-on*), la CPU si trova in uno stato di reset e inizia a eseguire istruzioni da un indirizzo fisico prefissato (Reset Vector). Poiché la RAM è volatile e vuota all'avvio, il primo programma eseguito è il **Bootstrap**, memorizzato stabilmente nella **ROM** (BIOS/UEFI). Questo software ha il compito di inizializzare l'hardware e caricare il sistema operativo dai supporti di memoria di massa.

L'Interfaccia di I/O e i Registri Ogni periferica comunica con il BUS attraverso un'interfaccia composta da una serie di **Registri di Interfaccia**. Questi sono tipicamente divisi in:

- **Registri Dati:** per lo scambio effettivo di informazioni.
- **Registri di Stato (Status Register):** per monitorare lo stato della periferica (es. bit *Ready* o *Error*).
- **Registri di Controllo (Control Register):** per inviare comandi alla periferica.

Effetti Collaterali (Side Effects)

A differenza della RAM, dove l'operazione di lettura è solitamente neutra, l'accesso ai registri di I/O può presentare **effetti collaterali irreversibili**. Ad esempio, la semplice lettura di un registro dati può causare lo svuotamento di un buffer interno o il reset automatico di un flag di interruzione (come il flag *FI* o *Done*). Si dice quindi che la lettura può essere *distruttiva*.

1.4 Mappatura dello Spazio di Indirizzamento

La CPU vede le periferiche come se fossero locazioni di memoria. Esistono due approcci principali alla gestione degli indirizzi:

1. **Memory-Mapped I/O (MMIO):** Le periferiche occupano una porzione dello spazio di indirizzamento della memoria RAM. La CPU usa le stesse istruzioni di lettura/scrittura (es. `MOV` o `LD/ST`) sia per la memoria che per l'I/O.

2. **Isolated I/O (Port-Mapped I/O):** Esiste uno spazio di indirizzamento separato per l'I/O, accessibile solo tramite istruzioni specifiche, come `IN` e `OUT`

Unicità dell'indirizzo e Collisioni Affinché il sistema funzioni correttamente, è necessario che la logica di decodifica degli indirizzi sia univoca.

- Se due periferiche (o una periferica e la RAM) avessero indirizzi sovrapposti, si verificherebbe un **conflitto sul bus dati**, rendendo l'informazione illeggibile o danneggiando i componenti.

- **Indirizzi non allocati:** Se la CPU tenta di leggere un indirizzo a cui non corrisponde né RAM né I/O, il bus rimane "appeso" (*floating*). In assenza di resistenze di pull-up/pull-down che forzano un valore (es. `0xFF`), il dato letto sarà casuale, frutto del rumore elettronico presente sulle linee.

Memoria Video Un caso particolare è la memoria video (VRAM), mappata direttamente nello spazio di indirizzamento della CPU. Scrivere in questa zona di memoria non altera solo dei dati binari, ma produce l'effetto collaterale visibile di cambiare i pixel sul monitor.

1.5 CPU - Il Ciclo Fondamentale (Fetch-Execute)

Il funzionamento della CPU è regolato da un ciclo continuo basato sul registro **IP** (*Instruction Pointer*, o *PC - Program Counter* in altre architetture). Questo registro contiene l'indirizzo di memoria della prossima istruzione da eseguire.

Il ciclo segue tre fasi principali:

1. **Fetch:** La CPU preleva l'istruzione dalla memoria all'indirizzo indicato da IP.

2. **Decode:** L'unità di controllo interpreta l'opcode dell'istruzione e i relativi operandi.

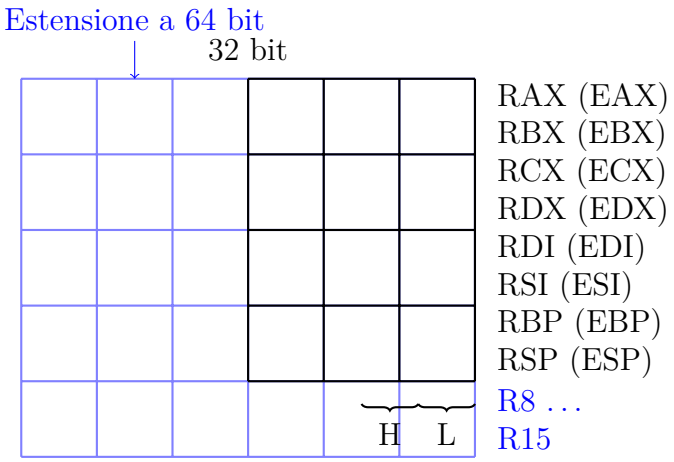
3. **Execute:** L'istruzione viene eseguita e l'IP viene incrementato per puntare all'istruzione successiva (o aggiornato in caso di salti/CALL).

1.6 L'Evoluzione a 64 bit (AMD64)

L'architettura a 64 bit estende la precedente IA-32 garantendo la retrocompatibilità (può eseguire codice a 32 bit). Le novità principali riguardano l'estensione della dimensione dei dati e l'aumento del numero di registri.

1.6.1 Organizzazione dei Registri

Nell'architettura a 64 bit, il prefisso **'E'** (Extended, 32 bit) viene sostituito dal prefisso **'R'** (64 bit). Inoltre, il numero di registri *General Purpose* viene raddoppiato, passando da 8 a 16.



Reminder: Tipologie di Operandi

L'architettura mantiene tre tipologie di operandi con la stessa sintassi dell'Assembler a 32 bit:

- **Immediato:** Un valore costante (es. `$0x10`).
- **Registro:** Accesso diretto ai registri (es. `%rax`).
- **Memoria:** Accesso tramite indirizzo (es. `(%rbx)`).

1.7 Indirizzamento Relativo e Limiti Fisici

Una delle novità più rilevanti è l'introduzione dell'indirizzamento relativo al registro IP: `offset(%rip)`.

Indirizzamento RIP-relative Questo metodo specifica quanto dista il dato dall'istruzione corrente (*RIP contiene già l'indirizzo della prossima istruzione*). Per ottimizzare la lunghezza delle istruzioni, gli **immediati** e gli **offset** rimangono solitamente a **32 bit**. Ciò significa che il salto o la distanza tra codice (`.text`) e dati (`.data`) non può superare i ±2 GB (il range di un intero a 32 bit con segno).

Istruzione MOVABS L'unica eccezione a questa regola è l'istruzione `MOVABS`.

- **Reminder:** Mentre la normale `MOV` supporta solo operandi a 32 bit (estesi poi a 64), `MOVABS` permette di caricare un valore immediato a 64 bit direttamente in un registro o di accedere a un indirizzo di memoria assoluto a 64 bit.

Vincolo dei 2GB

L'assemblatore deve conoscere preventivamente la posizione del programma. Poiché le istruzioni `CALL` e gli accessi ai dati tramite offset da `%rip` usano valori a 32 bit, le sezioni del programma devono essere "vicine" in memoria (distanza < 2GB).

1.8 Spazio di Indirizzamento e Indirizzi Canonici

Sebbene l'architettura AMD64 supporti teoricamente indirizzi a 64 bit (2^{64} byte, ovvero circa 16 Exabyte), le implementazioni attuali dei processori (Intel e AMD) utilizzano solitamente un'ampiezza di **48 bit** per gli indirizzi fisici e virtuali. Ciò consente di indirizzare fino a **256 Terabyte**.

Normalizzazione: La Forma Canonica Per garantire la compatibilità futura, l'architettura impone che ogni indirizzo sia in **forma canonica**. Poiché solo i primi 48 bit sono utilizzati, l'hardware richiede che i restanti 16 bit (dal 48 al 63) siano una copia esatta del bit 47 (*sign extension*):

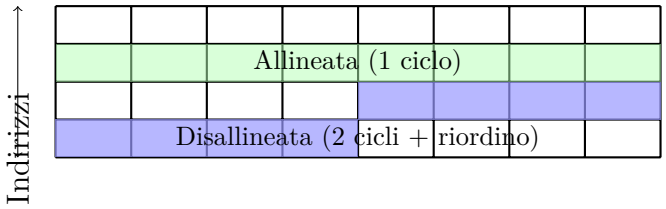
- Se il bit 47 è **0**, i bit 48-63 devono essere tutti **0**. Gli indirizzi validi vanno da `0x0000 0000 0000 0000` a `0x0000 7FFF FFFF FFFF`.
- Se il bit 47 è **1**, i bit 48-63 devono essere tutti **1**. Gli indirizzi validi vanno da `0xFFFF 8000 0000 0000` a `0xFFFF FFFF FFFF FFFF`.

Questo meccanismo crea un enorme "buco" di indirizzi non utilizzabili nel mezzo dello spazio di indirizzamento, dividendo la memoria in due tronconi (spesso usati per distinguere lo *User Space* dal *Kernel Space*). Se un programma tenta di usare un indirizzo non canonico, la CPU genera un'eccezione (*General Protection Fault*).

1.9 Allineamento dei Dati in Memoria

Un oggetto si definisce **allineato** se il suo indirizzo di partenza è un multiplo della sua dimensione (es. un intero da 8 byte allineato avrà un indirizzo A tale che $A \pmod 8 = 0$).

Il costo del disallineamento Possiamo immaginare la memoria a 64 bit come una sequenza di righe da **8 byte**. Il bus dati e la logica di controllo della CPU sono ottimizzati per leggere un'intera riga (o una sua parte contenuta internamente) in un unico ciclo di memoria.



Se una variabile di 8 byte è posta a cavallo tra due righe (come mostrato in blu nel diagramma):

1. La CPU deve effettuare **due accessi** distinti alla memoria per prelevare entrambe le righe.

2. È necessaria un'operazione interna di **shifting e masking** (riordino dei pezzi) per ricostruire il dato originale, poiché i byte arrivano al processore nelle posizioni sbagliate del bus.

Differenze tra Architetture

- **Intel (x86/x86-64):** Gestisce il disallineamento in hardware. Il programma funziona, ma subisce un calo di prestazioni dovuto ai cicli extra.

- **ARM / RISC:** Spesso sono più rigide. Un accesso disallineato può causare un *Alignment Fault*, interrompendo l'esecuzione del programma se non gestito esplicitamente dal software.

1.10 Rappresentazione dei dati: Endianness

Quando dobbiamo memorizzare un dato più grande di un singolo byte (ad esempio una *word* da 2 byte o una *quadword* da 8 byte), sorge il problema dell'ordine dei byte in memoria.

Consideriamo l'esempio del valore esadecimale `0x1122`:

- `11` è il byte più significativo (MSB - *Most Significant Byte*).

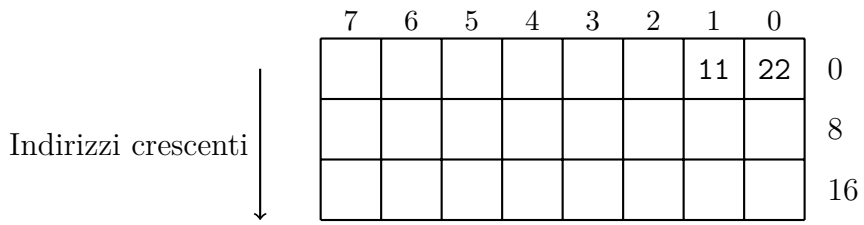
- `22` è il byte meno significativo (LSB - *Least Significant Byte*).

Esistono due strategie opposte:

1. **Big Endian:** Il byte più significativo (`11`) viene memorizzato all'indirizzo più basso. È l'ordine "naturale" di lettura (da sinistra a destra) ed è lo standard utilizzato nelle comunicazioni di rete (*Network Byte Order*).

2. **Little Endian:** Il byte meno significativo (`22`) viene memorizzato all'indirizzo più basso. È l'architettura adottata da **Intel** e AMD.

Analisi dello schema di memoria Nell'immagine, la memoria è rappresentata come una griglia dove ogni riga contiene 8 byte (64 bit). Gli indirizzi aumentano da destra verso sinistra all'interno della riga, o seguendo la numerazione delle righe (0, 8, 16...).



Come si nota, nel **Little Endian**, il valore appare "al contrario" se guardiamo la memoria: all'indirizzo 0 troviamo la parte "piccola" (`22`).

1.11 Nomenclatura e Potenze di 2

PIccolo reminder sui valori Esadecimali.

Esadecimale e Byte Ogni byte (8 bit) è rappresentato esattamente da **due cifre esadecimali** (es. `0xFF` = `1111 1111`). Questo perché una cifra esadecimale copre esattamente 4 bit ($2^4 = 16$).

Proprietà delle potenze di 2 Per i calcoli sugli indirizzi e sulle maschere di bit (bitmask), ricordiamo le seguenti forme notevoli in binario:

- 2^a: In binario è rappresentato da un `1` seguito da a zeri.

$$2^3 = 8_{10} = 1000_2$$

- $2^a - 1$: In binario è rappresentato da una sequenza di a uni. Questa forma è utilissima per creare filtri che "accendono" un certo numero di bit.

$$2^3 - 1 = 7_{10} = 111_2$$

Barbatrucco

Quando vedi un indirizzo esadecimale e devi capire se un oggetto è allineato a 2^n byte, basta guardare le ultime cifre:

- Allineato a 4 byte (2^2): l'ultima cifra esadecimale deve essere `0, 4, 8, C`.
- Allineato a 16 byte (2^4): l'ultima cifra esadecimale deve essere `0`.

1.12 Offset, Intervalli e Circolarità

L'indirizzamento della memoria si basa su concetti geometrici e algebrici precisi che permettono alla CPU di navigare tra i dati.

Offset e Intervalli L'**offset** rappresenta la distanza tra due indirizzi. Se abbiamo un indirizzo base x e vogliamo raggiungere un indirizzo y, l'offset è la quantità di indirizzi (byte) da "saltare".

- Per convenzione, si utilizzano intervalli **semiaperti a destra** $[x, y)$.
- Questa scelta è vantaggiosa perché la dimensione dell'intervallo è data semplicemente da $y - x$.
- Permette la **composizione** immediata: $[x, y) + [y, z) = [x, z)$. In questo modo, l'indirizzo di confine y viene contato una sola volta, evitando errori di tipo "off-by-one".

Circolarità dello Spazio Lo spazio di indirizzamento è **circolare** (aritmetica modulare $|\cdot|_{2^n}$). Questo è dovuto alla natura fisica della CPU:

- Se abbiamo un bus indirizzi a n fili, possiamo rappresentare solo 2^n valori.
- Sommando 1 all'indirizzo massimo $(2^n - 1)$, si otterrebbe un numero con un '1' nell'$(n + 1)$-esimo bit (il *carry*). Poiché abbiamo solo n fili, quel bit viene scartato e l'indirizzo torna a `0`.

Gestione del Segno negli Offset

- Se l'offset è espresso su n bit (la stessa dimensione dell'indirizzo), la circolarità rende irrilevante la distinzione tra positivo e negativo grazie alla rappresentazione in **Complemento a 2**.
- Se l'offset è espresso su un numero di bit **inferiore a n** (es. un immediato a 12 bit in un'istruzione a 64 bit), è fondamentale conoscere il segno. In questo caso, la CPU effettua la **estensione del segno** per preservare il valore numerico corretto durante la somma con l'indirizzo base.

1.13 Confini (Boundaries) e Regioni Naturali

Lo spazio di indirizzamento viene suddiviso in blocchi di dimensione pari a potenze di 2 (2^b).

Definizione di Boundary Un indirizzo si dice trovarsi su un **confine** (*boundary*) di 2^b se è un multiplo esatto di 2^b.

- **Proprietà binaria:** Un indirizzo al confine di una porzione di 2^b byte avrà sempre i b bit meno significativi (LSB) pari a **zero**.

Regioni Naturali Una **regione naturale** di dimensione 2^b è l'insieme degli indirizzi compresi tra due confini consecutivi di 2^b. Per identificare un indirizzo all'interno di una geografia di regioni, dividiamo i suoi n bit in due parti:

1. **n-b bit più significativi (MSB):** Rimangono **costanti** per tutti gli indirizzi all'interno della stessa regione. Essi rappresentano il "nome" o l'indice della regione (il "regime naturale" citato nei tuoi appunti).

2. **b bit meno significativi (LSB):** Variano da 0 a $2^b - 1$. Rappresentano l'**offset** (la posizione relativa) all'interno di quella specifica regione.

$$\underbrace{\boxed{\text{Indice Regione } (n-b \text{ bit}) \mid \text{Offset } (b \text{ bit})}}_{\text{Costanti nella regione}} \quad \text{Variano } 0 \ldots 2^b - 1$$

Esempio Pratico: Pagine da 4KB

Se le regioni sono pagine da $4KB$ (2^{12}), allora $b = 12$.

- Gli ultimi 12 bit dell'indirizzo indicano dove si trova il dato nella pagina.
- I restanti bit in alto indicano quale pagina stiamo leggendo.

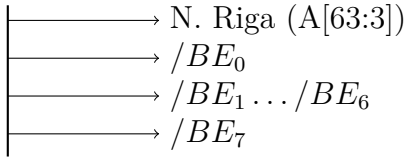
1.14 L'Interfaccia Fisica: Bus Dati e Byte Enable

Fisicamente, la memoria non è un array di singoli byte, ma è organizzata in **righe da 8 byte** (64 bit), coerentemente con la larghezza del bus dati.

Quando la CPU deve accedere a un dato, non invia l'indirizzo esatto del singolo byte sul bus indirizzi, ma opera in questo modo:

- **Indirizzo di Riga:** Invia i bit più significativi dell'indirizzo, che identificano la riga da 8 byte (corrispondenti a un indirizzo multiplo di 8).

- **Byte Enable ($/BE_0 \ldots /BE_7$):** Utilizza 8 segnali di controllo dedicati per indicare quali degli 8 byte all'interno di quella riga devono essere effettivamente letti o scritti.



Segnali emessi dalla CPU

Esempio: Lettura disallineata Supponiamo di voler caricare un valore a 32 bit (4 byte) in `%eax` partendo dall'indirizzo **7**. Questo dato si trova a cavallo tra due righe fisiche:

1. **Ciclo 1 (Riga 0):** La CPU richiede la riga che inizia all'indirizzo 0, attivando solo il segnale $/BE_7$.

2. **Ciclo 2 (Riga 1):** La CPU richiede la riga successiva (indirizzo 8), attivando i segnali $/BE_0, /BE_1, /BE_2$.

3. **Riallineamento:** L'unità di controllo interna della CPU deve traslare (shift) e combinare i frammenti ottenuti per ricostruire il dato originale prima di memorizzarlo nel registro.

Questa architettura spiega perché l'allineamento dei dati è cruciale per le prestazioni: un accesso allineato richiede un solo ciclo di bus, mentre uno disallineato ne richiede almeno due più il tempo di elaborazione interna.

1.15 L'Organizzazione Fisica della Memoria: Righe e Byte Enable

A livello logico, pensiamo alla memoria come a una lunga sequenza di byte individuali. Tuttavia, a livello **fisico**, la CPU comunica con la RAM tramite un bus dati a 64 bit. Questo significa che la memoria è organizzata in **righe da 8 byte** ciascuna.

Il Bus Indirizzi e i segnali $/BE$ Quando la CPU emette un indirizzo A per leggere un dato, il controller di memoria scompone l'operazione:

- **Indirizzo di Riga ($A[63:3]$):** I bit più significativi dell'indirizzo indicano *quale* riga da 8 byte deve essere selezionata (es. Riga 0, Riga 8, Riga 16...).

- **Byte Enable ($/BE_0 \ldots /BE_7$):** Sono 8 segnali fisici (fili) che fungono da "interruttori" per i singoli byte all'interno della riga selezionata.

Se la CPU vuole leggere solo il terzo byte della riga 0, invierà l'indirizzo di riga 0 e attiverà solo il segnale $/BE_2$. Se vuole leggere un intero a 32 bit (4 byte) allineato, attiverà contemporaneamente $/BE_0, /BE_1, /BE_2, /BE_3$.

1.16 Analisi di una Lettura Disallineata

Il problema sorge quando un dato di N byte **attraversa il confine** tra due righe fisiche. Riprendiamo l'esempio di una `MOV` di un dato a 32 bit (4 byte) a partire dall'indirizzo **7**.

Indirizzo	**+0**	**+1**	**+2**	**+3**	**+4**	**+5**	**+6**	**+7**
Riga 0	byte 0	byte 1	byte 2	byte 3	byte 4	byte 5	byte 6	**Dato[0]**
Riga 8	**Dato[1]**	**Dato[2]**	**Dato[3]**	byte 11	byte 12	byte 13	byte 14	byte 15

Tabella 1: Esempio di dato a 32 bit memorizzato all'indirizzo 7.

Poiché il bus può accedere a **una sola riga per ciclo**, la CPU deve eseguire due operazioni distinte:

1. **Ciclo 1 (Accesso alla Riga 0):**
   - La CPU richiede la riga che inizia all'indirizzo 0.
   - Attiva solo $/BE_7$.
   - Riceve il primo byte del dato (*Dato[0]*) e lo salva in un registro temporaneo interno.
2. **Ciclo 2 (Accesso alla Riga 1):**
   - La CPU richiede la riga che inizia all'indirizzo 8.
   - Attiva i segnali $/BE_0, /BE_1, /BE_2$.
   - Riceve i restanti tre byte (*Dato[1,2,3]*).

Conclusione: Shifting e Prestazioni Una volta ottenuti i due pezzi, la CPU deve "shifttare" (scorrere) i bit internamente per riallinearli e formare il numero a 32 bit corretto.

- **Flessibilità:** I segnali $/BE$ permettono di leggere qualsiasi combinazione di byte, rendendo possibile il disallineamento.
- **Costo:** Una lettura disallineata è intrinsecamente più lenta perché richiede **due cicli di bus** anziché uno solo, oltre a un lavoro extra di manipolazione dei bit all'interno della CPU.